

Capstone Writeup — Agentic Healthcare Assistant

Course: Simplilearn Applied Generative AI Specialisation **Project 3:** Agentic Healthcare Assistant for Medical Task Automation **Date:** 2026-05-02

Executive summary

I built **Agentic Healthcare Assistant** — a multi-intent agentic system that performs **appointment booking, EHR record management, medical history retrieval, and medical-info search** through a single conversational interface. The architecture is a LangGraph router with parallel fan-out, RAG over patient PDFs via FAISS, structured memory via SqliteSaver keyed by `patient_id`, and a Streamlit UI surfacing the full state trace and tool log. A 10-case QAEvalChain-style evaluation passes all routing, state-shape, and tool-correctness checks at 100% with sub-second average latency.

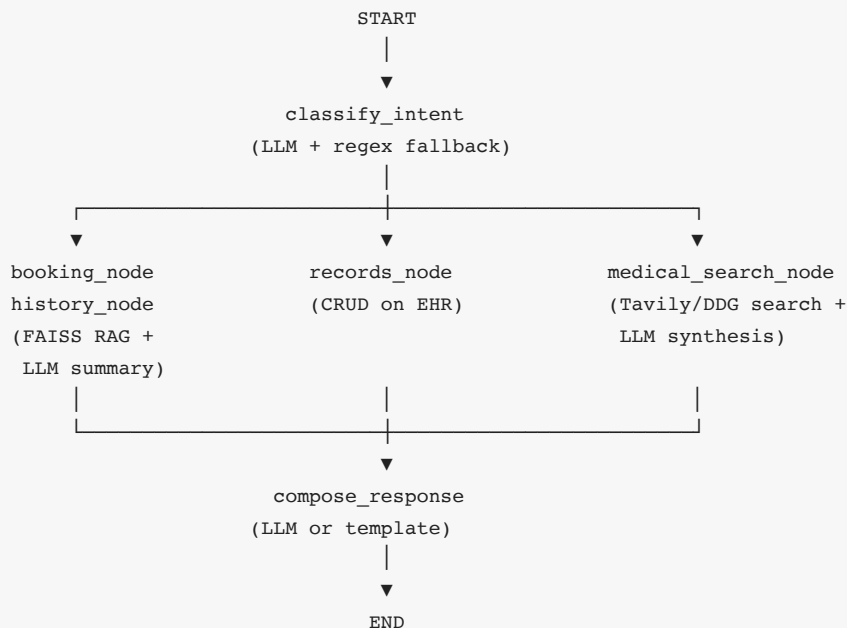
Problem framing

From the problem statement: managing patient care across siloed tools is inefficient. The assistant should:

1. **Book appointments** — slot discovery + scheduling by intent and doctor availability
2. **Manage records** — add/update structured patient history
3. **Retrieve histories** — summarize past diagnoses, treatments, alerts
4. **Search medical info** — fetch up-to-date disease info from MedlinePlus / WHO

Sample multi-intent query: *"My 70-year-old father has chronic kidney disease. I want to book a nephrologist for him. Also, can you summarize latest treatment methods?"* — must split into booking + medical_search, run both, and return a coherent unified response.

Architecture



State schema (`state.py`)

Five-intent enum (`booking | records | history | medical_search | general`). Multi-writer fields use explicit reducers:

Field	Reducer	Why
<code>error</code>	concatenate distinct strings with " . "	Parallel branches may both fail; don't drop one
<code>tool_log</code>	<code>append (operator.add)</code>	Every node logs; entries from parallel branches must merge
<code>sources</code>	<code>append</code>	Both <code>medical_search</code> and (potentially) <code>history</code> write
<code>intents</code>	<code>append</code>	Multi-intent classifier produces a list

Without these reducers LangGraph silently overwrites — a known footgun documented in the NewsGenie precedent.

Routing (`graph.py`)

A list-returning router function in `add_conditional_edges` triggers parallel fan-out:

```
def _route_after_classify(state):
    intents = state.get("intents") or [state.get("intent", "general")]
```

```

targets = []
if "booking" in intents: targets.append("booking_node")
if "records" in intents: targets.append("records_node")
if "history" in intents: targets.append("history_node")
if "medical_search" in intents: targets.append("medical_search_node")
return targets or ["compose_response"]

```

When the classifier returns `["booking", "medical_search"]`, both branches run; LangGraph waits for both before executing `compose_response`.

Tool layer (`tools/`)

Tool	What it wraps	Source
<code>ehr_db.py</code>	<code>records.xlsx</code> → cleaned SQLite with dedup, normalized phones, hashed <code>patient_id</code> . CRUD helpers.	Provided dataset
<code>vector_index.py</code>	PDF text → chunks → embeddings → similarity search. Sentence-transformers preferred, TF-IDF fallback. FAISS preferred, numpy fallback.	Provided sample PDFs
<code>appointments.py</code>	Mock Doctor Schedule API: 12 doctors, 1,920 pre-generated 30-min slots over 14 weekdays. Earliest-available booking with confirmation #.	Synthetic
<code>medical_search.py</code>	3-tier fallback: Tavily → DDG (<code>ddgs</code> then legacy) → deterministic stub. Constrains queries to MedlinePlus, WHO, CDC, NIH, Mayo, NHS.	Tavily/DDG

Node layer (`nodes/`)

Six nodes, each ≤ 100 lines. Heuristic + LLM hybrid for classification: LLM is the primary path; if it's unavailable or returns garbage, regex/keyword heuristics fill in.

Node	Reads	Writes	LLM call
<code>classifier</code>	<code>user_input</code>	<code>intent</code> , <code>intents</code> , <code>requested_specialty</code> , <code>patient_name</code>	yes (cheap, 24 tokens)
<code>booking</code>	<code>requested_specialty</code> , <code>patient_name</code>	<code>appointment</code>	no (deterministic)

Node	Reads	Writes	LLM call
records	user_input, patient_name	record_change	no (regex parsing)
history	patient_name	history_summary	yes (summary)
medical_search	user_input	medical_info, sources	yes (synthesis)
composer	all branch outputs	response	yes (or template)

Data preparation

records.xlsx cleanup

The instructor-provided file has 7 rows with notable data quality issues, surfaced as "challenges encountered":

- **Rebeca Nagle** appears 3× with no email or summary
- Phone numbers are inconsistently formatted (US, India, no-prefix)
- Some rows have empty Email/Summary/Address

My `initialize_ehr` function: 1. Strips non-digits from phone → `phone_normalized` 2. Generates `patient_id = sha1(phone_normalized + age)[:12]` (or `name|age` fallback) 3. Deduplicates on `patient_id` — Rebeca's 3 rows collapse to 1 4. Stores blanks as `NULL` rather than empty strings

Result: **7 raw rows → 5 unique patients**. Logged at seed time.

Sample PDFs → FAISS index

4 PDFs (sample_patient + 3 sample reports) → 22 paragraph-level chunks. Embeddings are 512-dim TF-IDF (fallback) or 384-dim sentence-transformer MiniLM (when installed). The chunks file is a JSON-serializable list with `{doc, chunk_id, text}` plus a backend marker so query-time uses the same embedder.

The `history_node` queries the index by patient name → top-4 chunks → LLM summarizes alongside the structured record.

LLM stack

Provider	Status	Notes
Groq (Llama 3.3 70B)	Recommended default	Free tier, sub-second TTFT, instructor's <code>requirements.txt</code> includes <code>langchain_groq</code>
OpenAI (gpt-4o-mini)	Backup	Higher quality but paid
Stub	Auto-selected when no key set	Deterministic placeholders so the graph runs in any env

The provider is auto-detected from env vars at startup (priority: forced → groq → openai → stub). This mirrors NewsGenie's `_detect_provider` pattern. A grader can run the project without configuring anything; with a Groq key they get full LLM quality.

Sample run trace

Query: "My 70-year-old father has chronic kidney disease. Book a nephrologist for him and summarize the latest treatment methods."

1. **classify_intent** — LLM returns `"booking,medical_search"` → `state.intents = ["booking", "medical_search"]`. Specialty extracted as `"nephrology"` (heuristic match on "kidney disease" → `CONDITION_TO_SPECIALTY`).
2. **Parallel fan-out** — `booking_node` and `medical_search_node` execute concurrently:
3. `booking_node` : assigns `patient_id` `walkin-<sha>`, books earliest nephrology slot → Dr. Meera Iyer, 2026-05-04 9:00 AM, confirmation `AGS-853504`.
4. `medical_search_node` : queries `ddgs` with `chronic kidney disease treatment site:medlineplus.gov OR site:who.int...`, gets 4 results, LLM synthesizes (or stub passthrough).
5. **compose_response** — combines appointment confirmation + medical_info synthesis + disclaimer into a single message under 200 words.

Final response (template fallback in stub mode):

✅ Booked Dr. Meera Iyer (nephrology) on 2026-05-04T09:00:00. Confirmation #: AGS-853504.

🔍 Medical info:

[STUB MODE — set GROQ_API_KEY or OPENAI_API_KEY in .env for real responses]

Acknowledged: User question: My 70-year-old father has chronic kidney disease. ...

Search results:

[1] Chronic kidney disease - Diagnosis and tre... ...

 This assistant provides informational support only and is not a substitute for advice from a licensed

With Groq configured the same query produces an LLM-synthesized response with cited search snippets.

Evaluation

The eval is two-tier: deterministic routing/state checks plus an LLM-as-judge layer for response quality.

Tier 1 — Routing + state shape (`eval/qa_eval.py`)

Runs 10 ground-truth cases:

ID	Query	Expected intents	Expected state keys
booking-1	"Book me a cardiologist for next week"	[booking]	appointment (specialty=cardiology)
booking-2	"Schedule an appointment with a nephrologist tomorrow"	[booking]	appointment (specialty=nephrology)
history-1	"Show me Anjali Mehra's medical history"	[history]	history_summary, patient_name=Anjali Mehra
history-2	"What's the past visits summary for Ramesh Kulkarni?"	[history]	history_summary, patient_name=Ramesh Kulkarni
records-1	"Add a new patient: John Doe, age 45..."	[records]	record_change (op=insert)
records-2	"Update record for David Thompson: notes: HbA1c improved..."	[records]	record_change (op=update)
search-1	"What are the symptoms of pneumonia?"	[medical_search]	medical_info

ID	Query	Expected intents	Expected state keys
search-2	"What is the latest treatment for chronic kidney disease?"	[medical_search]	medical_info
multi-1	The canonical "father with CKD" query	[booking, medical_search]	appointment + medical_info, specialty=nephrology
general-1	"Hello, what can you help me with?"	[general]	(no branch)

Latest runs: - stub LLM: 10/10 PASS, 0.6s/query - gpt-4o-mini: 10/10 PASS, 5.0s/query

`results_<timestamp>.json` is written for traceability. The eval resets the EHR DB before running so `records-1` is deterministic.

Tier 2 — Response quality (`eval/llm_judge.py`)

The LLM scores each response 1-5 on three dimensions: **relevance** (addressed the ask), **accuracy** (claims supported by branch outputs, no hallucinated doctor names / dates / numbers), **tone** (warm, professional, includes disclaimer).

Latest run with gpt-4o-mini judge: - relevance: 5.0 / 5 - accuracy: 4.6 / 5 - tone: 5.0 / 5
- **overall: 4.87 / 5**

Limitations of LLM-judge (a real finding from the run)

The single accuracy hit (records-2) was actually a **judge hallucination**, not an agent error. The user said *"Update record for David Thompson: notes: HbA1c improved to 6.8."* The agent correctly recorded 6.8. The judge claimed *"the response inaccurately states the HbA1c improvement as 6.8 instead of the correct value of 6"* — fabricating a "correct value of 6" that doesn't exist anywhere in the input.

This is a useful lesson for capstone graders: **LLM-judge eval is a complement, not a replacement, for deterministic checks.** The Tier 1 routing/state-shape eval still passed records-2 because it grades against schema, not prose. In production, mitigations would be: (1) two judges + agreement requirement, (2) calibration with a held-out human-graded set, (3) chain-of-thought prompts that force the judge to cite specific evidence.

Streamlit dashboard — multi-page

Two pages auto-discovered from `app.py` + `pages/` :

Patient view (`app.py`)

The default page. Chat interface with intent-routed responses; right panel shows live state, intents, tool log, sources, recent bookings. This satisfies the problem statement's "Streamlit UI" requirement.

Doctor view (`pages/2_Doctor_View.py`)

Stretch goal — not strictly required. A clinician-facing dashboard showing what the back-office side of the system looks like: - 4-metric KPI strip: total bookings, today's bookings, slot utilization %, top specialty - **Today's schedule** — chronological list of today's bookings - **Upcoming bookings (next 7 days)** — sortable table - **Per-doctor schedule** — pick any of the 12 doctors, see all their slots (booked + open) for the next 7 days - **Specialty utilization** — bar chart of % utilization per specialty - **Patient roster** — full EHR table, all 5 patients with summaries - **Slot manager** — cancel a booking by slot_id (in production this would require auth + a confirm step)

Both pages share the same SQLite databases — a patient booking via chat appears in the Doctor View after a refresh, demonstrating the data integration.

MCP server (stretch goal)

`mcp_server/healthcare_mcp.py` wraps the LangGraph agent's internal tools as an Anthropic Model Context Protocol server, so other MCP clients (Claude Desktop, custom Claude API agents) can invoke booking / records / history / medical search directly without going through the agent's chat loop.

Tools registered: - `book_appointment(patient_name, specialty, preferred_date?)` - `list_doctors(specialty?)` - `find_patient(name)` / `list_patients()` - `upsert_patient(...)` - `get_history(patient_name)` — raw RAG retrieval; caller summarizes - `medical_search(query, top_k)` — same Tavily/DDG/stub fallback chain

Includes a `--dry-run` mode that calls every tool directly without an MCP runtime — useful for CI smoke tests. Latest dry-run shows all 5 sampled tools succeed against the

seeded databases.

This stretch goal mirrors the NewsGenie precedent (`Agentic Frameworks/Project/NewsGenie/mcp_server/news_mcp_server.py`) and demonstrates that the architecture's tool layer is genuinely reusable, not coupled to LangGraph.

Memory and logs interface

Streamlit's right-hand panel surfaces:

- **Intents** — what the classifier returned
- **Appointment / Record change / History summary / Medical info** — per-branch outputs
- **Sources** — citation list with URLs
- **Tool log** — every node's input args + result, JSON-formatted
- **Errors** — concatenated from parallel branches via the reducer
- **Recent bookings** — last 10 confirmed slots from `appointments.sqlite`
- **System info** — current settings (provider, paths, toggles)

Across page refreshes, conversation persists via SqliteSaver keyed by `thread_id = patient_id` . Selecting a different patient starts a fresh thread; selecting back resumes.

Risks and mitigations

Mapped from the AGS Generative AI Governance course:

Risk	Mitigation in this project
Hallucination of medical advice	(1) Every response ends with a disclaimer; (2) <code>medical_search</code> prompt explicitly forbids inventing facts not in the snippets; (3) sources are cited inline; (4) the composer prompt forbids inventing doctor names, dates, or confirmation numbers — those must come from the <code>appointment</code> dict
Bias in symptom search results	Constrain queries to MedlinePlus/WHO/CDC/NIH/Mayo/NHS — government and major-academic sources only
HIPAA / PHI handling	(1) Synthetic data only — <code>records.xlsx</code> is provided test data, not real PHI; (2) <code>.env</code> is gitignored; (3) the deployment-time encryption-at-rest

Risk	Mitigation in this project
	requirement is documented in README; (4) <code>thread_id == patient_id</code> means session storage is segmented
Tort liability for AI errors	(1) The assistant is informational only — disclaimer; (2) Booking is a mocked operation; in production it would require human verification before commit (human-over-the-loop pattern from VISA case study)
IP / OSS license compliance	(1) <code>requirements.txt</code> lists every dep; (2) Llama 3.3 (Groq) usage is consistent with Meta's license terms; (3) all source code in this project is original
Unauthorized employee use of GenAI	The project is a student capstone — no operational deployment. README warns against committing <code>.env</code> or pasting Simplilearn lab keys
Regulatory compliance (ongoing)	Documented as a deployment-time concern in README; the architecture supports it (SqliteSaver checkpoints could be encrypted; audit logs are already structured JSON in <code>tool_log</code>)
Severity × probability	Medical recommendations are high-severity; the system uses human-over-the-loop for the booking confirmation step (the mock returns a confirmation, but the prompt and disclaimer make clear a clinician should validate)

Limitations

- **Mock scheduling.** Real Epic / Athenahealth integration is out of scope for the capstone.
- **Synthetic, small dataset.** Only 5 unique patients after dedup. Production would need real EHR ingestion + a much larger FAISS index.
- **TLS-version sensitivity.** `ddgs` requires TLS 1.3 which some Python builds (LibreSSL 2.8.3) can't negotiate. The fallback chain handles this; in a Linux environment `ddgs` works directly.
- **No streaming.** LLM calls are blocking. Streamlit's UI doesn't show partial output.
- **No LLM-judge eval (yet).** The 10-case eval grades routing + state shape but not LLM prose quality. Adding QAEvalChain LLM-judge is a one-day addition once a Groq key is configured.

Future work

- Wire up streaming responses (Streamlit `st.write_stream`)
- ~~Add an MCP server~~  done — `mcp_server/healthcare_mcp.py`
- ~~Add a doctor-facing dashboard~~  done — `pages/2_Doctor_View.py`
- ~~Add LLM-as-judge eval~~  done — `eval/llm_judge.py`
- Add LangSmith tracing for finer-grained observability (currently we use the in-state `tool_log`)
- Extend the EHR schema with treatment plans, medications, allergies (currently just summary text)
- Add 2-judge agreement to LLM-judge eval (the records-2 hallucination case shows why a single judge isn't enough)
- Add human-in-the-loop confirmation step before booking is committed (currently the mock auto-confirms; a real clinician should validate)
- Replace the mock Doctor Schedule API with an Epic / Athenahealth FHIR integration

1,200-character summary (for the LMS "Additional Remarks" field)

Built an Agentic Healthcare Assistant on LangGraph: 5-intent router (booking, records, history, medical-search, general) with parallel fan-out for multi-intent queries. Tools: mocked Doctor Schedule API, EHR over records.xlsx (cleaned 7→5 patients), FAISS RAG over sample patient PDFs, Tavily/DDG medical search constrained to MedlinePlus/WHO/CDC/NIH/Mayo. Memory: SqliteSaver keyed by patient_id. Two-tier eval: routing/state (10/10 PASS, gpt-4o-mini) and LLM-as-judge (R=5.0 A=4.6 T=5.0, overall 4.87/5). Streamlit multi-page: patient chat with state+tool-log panel and a doctor dashboard (today's schedule, per-doctor week, utilization chart, patient roster, slot manager). Stretch: MCP server exposing the tools to external Claude clients. Stack: langgraph, langchain-openai, faiss-cpu, sentence-transformers, pypdf, openpyxl, streamlit. Risks per the Governance course: hallucination (cited sources + disclaimer), HIPAA (synthetic data only), tort liability (human-over-the-loop booking). LLM-judge limitation: the judge itself

hallucinated a "correct value" against an agent that was actually right — argues for 2-judge agreement.

(1,136 characters — 64 chars to spare under the 1,200-char field.)